

Code-Layout, Readability and Reusability

Date	2 October 2024
Team ID	SWTID-2026-6275 (Team Cipher)
Project Name	LegalEase: AI-Powered Legal Document Generator
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	4-space PEP8 indentation in Python; 2-space in HTML/JS.
2	Proper File Structure	Files and folders are logically organized	Yes	Separate folders: routes, services, templates, static, models.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Descriptive names such as document_type, user_inputs.
4	Function / Method Names	Functions are descriptively named	Yes	e.g. generate_document(), export_to_pdf(), validate_form().
5	Code Comments	Inline and block comments explain logic	Partial	Core logic has comments and docstrings; some helpers pending.
6	Modular Design	Code is split into reusable functions/modules	Yes	Auth, AI, export and template modules are independent.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Shared helpers extracted; unused code cleaned up.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	try/except around AI API, DB and file export with user messages.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	AI Generation Service (prompt builder)	Python / LLM API	All document templates	High

2	Template Engine	Python / Jinja2	Every document type	High
3	Form Validation Components	JavaScript / HTML	All input forms	High
4	PDF / DOCX Export Utility	Python	All generated documents	Medium
5	Authentication Module	Python / Flask	Login, dashboard, history pages	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear modular structure with separate layers.
Readability	4	Meaningful names and consistent formatting.
Reusability	4	Core services reused across all document types.
Documentation / Comments	3	Docstrings present for main modules; can be extended.
Overall Score	4	Well-organised, maintainable codebase.